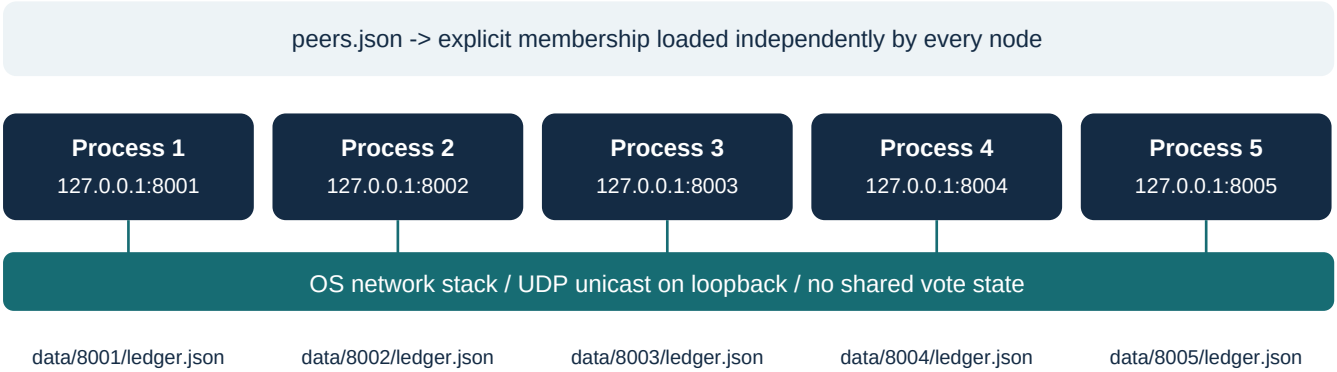
1. From color demo to data consensus

The project refactors the supplied peer.py into a small networked application that agrees on one structured data block. It retains leaderless random sampling, an agreement threshold, and repeated confidence building. It deliberately does not implement mining, a mempool, account balances, or a multi-block blockchain.

Starter shortcuts and concrete changes

Starter behavior	Implemented replacement
Broadcast HELLO on a shared discovery port	Static peers.json; one UDP socket per node; every datagram has a specific destination.
Color strings as preferences	Full candidate blocks validated with SHA-256; preferences identified by block hashes.
LAN-oriented address discovery	Explicit 127.0.0.1 addresses and five distinct ports; independent OS processes.
Remote finalization accepted immediately	Each node reaches its own beta threshold, writes its own ledger, and continues answering queries.

Architecture



Within each process, a listener thread receives messages while a sampling thread runs the consensus loop; the main thread handles commands. A lock protects shared in-process protocol state. No node shares its blocks, votes, or confidence with another process through Python objects, files, or memory.

Scope and implementation choices

Each node proposes index 1 with an all-zero previous hash and two generated transaction strings. The starter did not include transactions.csv, so generation follows the assignment's permitted alternative. The launcher coordinates startup and observes output only. The protocol itself has no leader or central vote collector.

The code uses the Python standard library only. Terminal animation and repeated color elections were removed to keep the final single-decision application small and its logs readable.

2. Payload integrity and messages

Candidate block and canonical hashing

A block contains index, timestamp, proposer, prev_hash, transactions, and block_hash. The hash covers every field except block_hash itself. JSON keys are sorted, compact separators are fixed, non-ASCII text is preserved, and the result is encoded as UTF-8 before SHA-256. Array order remains meaningful.

```
fields = {k: v for k, v in block.items() if k != "block_hash"}
data = json.dumps(fields, sort_keys=True,
                  separators=(",", ":"), ensure_ascii=False)
block_hash = hashlib.sha256(data.encode("utf-8")).hexdigest()
```

On reception, the node validates the exact field set, supported index and previous hash, timestamp type, configured proposer, and bounded nonempty transaction list. It recomputes the digest before caching a block or counting a response. Invalid blocks are logged and ignored. Unknown network senders are ignored. Hash integrity does not authenticate an author or validate a payment.

Message protocol

Type	Fields and action
BLOCK_PROPOSAL	type, block. Share a complete candidate during bootstrap.
BLOCK_QUERY	type, query_id, block. Ask a randomly sampled peer for its current preference.
BLOCK_RESPONSE	type, query_id, block. Return the full preferred block, preserving the request ID.
BLOCK_FINALIZED	type, block. Announce a local decision. The recipient validates/caches the block but does not finalize from this notice.

Example exchange

Node 8001 sends BLOCK_QUERY with a fresh UUID and its preferred block to 127.0.0.1:8002. Node 8002 validates the supplied block and replies from its own socket with BLOCK_RESPONSE, the same UUID, and its own preferred block. Node 8001 validates that block and counts the hash only if 8002 was selected for this active query.

Responses are stored by sender address, so duplicates cannot add votes. The active query entry is removed after the timeout; an old response cannot enter a later round. The receiver checks the actual datagram source against configured membership, but this is not cryptographic authentication.

Persistence

Each node writes a one-element array to data/<port>/ledger.json. A temporary file is renamed over the target after a complete JSON write. A fresh process run is a new independent decision and replaces its previous output only on finalization. There is no chain recovery and no fsync-based crash-durability guarantee.

3. Consensus and local deployment

Sampling loop

Parameter	Value / behavior
K / alpha / beta	3 sampled neighbors / 2 identical responses / 10 consecutive successful rounds
Query timeout / interval	1.5 seconds collecting replies, then 1.0 second between attempts
Initial exchange	Four proposal announcements over two seconds after /start

The initial preference is the smallest hash among valid proposals observed during bootstrap. This explicit tie-break helps avoid a starting state with five unique preferences and no repeated response. It is an application-level initialization rule, not the consensus decision. Packet loss or delayed startup can still produce different initial views; no reliable all-node barrier is claimed.

Each attempt samples up to K distinct configured neighbors. If at least alpha responses agree, the node adopts that hash. Matching the previous preference increments confidence; changing preference sets confidence to 1. An unsuccessful round resets confidence to 0. Beta successful consecutive rounds cause local finalization. The counter includes every sampling attempt, including failed ones.

Reproducible five-process deployment

```
python3 run.py --demo
```

Run from the extracted project directory with Python 3.9+ installed and ports 8001-8005 free. No dependency installation is needed. The command list in run.py launches five separate peer.py processes, each using peers.json. The launcher waits for READY output, sends /start to each, checks saved ledgers against final hashes, and reports PASS only if all launched nodes agree. Normal local completion takes about 30 seconds.

Without --demo, the launcher accepts commands such as all /start, all /status, and /quit. Commands for individual nodes can also be run in separate terminals. Every node has its own IP:port, address space, state, and output file. Loopback datagrams traverse the OS network stack; local placement does not turn the nodes into a shared-memory simulation.

Absent-node behavior

Keep all five configuration entries but remove the fifth launch command. Four processes still sample from four configured neighbors, one of which is absent. A query waits only 1.5 seconds. A sample of three can still contain two live agreeing peers and satisfy alpha=2. The threshold is never lowered because of failures. If agreement is insufficient, confidence resets and another sample is attempted.

The node loop has no universal termination guarantee under arbitrary failure. The launcher separately limits observation to 120 seconds and reports a failure on timeout. Finalized nodes remain available for responses until the launcher stops all children after observing all decisions.

4. Experiments and limitations

Observed results

The recorded experiments used real subprocesses and UDP sockets. Every running node generated a distinct candidate block; the bootstrap rule reconciled the observed candidate set before the confidence phase. Thus these are conflicting-proposal experiments, not evidence of difficult split-preference recovery after bootstrap.

Experiment	Observed outcome
5 running / 5 configured Ports 8001-8005	Five distinct proposals; all nodes finalized at round 10 with their own confidence=10/10. Five saved blocks had identical valid hashes.
4 running / 5 configured 19005 never started	Four distinct proposals; ports 19001-19004 each finalized at round 10. The absent address remained configured. All four ledgers matched.
Tampered transaction payload	Original digest retained after modifying a transaction. Receiver rejected the block and did not count the response.
Automated verification	Eight tests passed: six targeted data/protocol checks and two separate-process integration scenarios. Recorded duration: 57.654 seconds.

The default-port run selected the block proposed by 127.0.0.1:8005. Full hash:

`211b125162700d8bef8d524fbf69cd3a523b9e63aea6bace3161a0fceaefe844`

Evidence: evidence/demo.txt, evidence/tests.txt, per-node logs and ledgers in evidence/five-nodes/ and evidence/four-nodes/, and screenshots of the recorded outputs. Reproduce the tests with python3 -B -m unittest -v test_peer. Rounds and timings are observations from these runs, not guarantees.

Limits of the model

This is a classroom Avalanche-inspired protocol, not production Snowball and not a proof of Byzantine safety. The small sample threshold is not a global quorum. A prolonged partition may cause conflicting local finalizations; neither a 99.9% guarantee nor a specific Byzantine-fault percentage is established here.

Membership is static; messages are unauthenticated; UDP delivery and ordering are not guaranteed. The minimum-hash initialization is predictable and can be manipulated by proposers. Late startup and incomplete proposal exchange can reduce convergence. Transactions are illustrative strings, and the hash only checks their internal integrity. A host-wide failure can stop every local process.

Next hardening step

Add authenticated messages and an explicit, retried bootstrap protocol with a bounded membership view. Then define and test the safety and liveness assumptions under packet loss and partitions. For larger payloads, transmit hashes during sampling and fetch validated blocks on demand. These extensions are intentionally outside this one-block submission.

Development note: the implementation, checks, and documentation were completed with AI assistance after an initial guided walkthrough.